Drishya

From an idea to a visual explanation. Built to run locally.

Application documentation

Product guide, architecture, local setup and demonstration workflow. Prepared for the 24-hour Blu Visual Teacher Hackathon.

Document snapshot	October 28, 2026
Generation engine	Qwen3-8B + AI4Bharat Indic Parler TTS
Video engine	Authored Remotion components + FFmpeg
Demo laptop	MacBook Pro / Apple M5 / 16 GB unified memory

What Drishya does

Drishya turns a typed concept, process description or uploaded document into a narrated video. A chat workspace collects preferences, coordinates generation and saves conversations, revisions and completed videos in a library. Local models provide lesson data and speech; application-authored components provide the animation.

Guide contents

1. Product and user journey
2. System architecture
3. Visuals, narration and export
4. Installation and configuration
5. Offline operation
6. Documents, quizzes and sharing
7. Persistence and operations
8. Demo, hardware and licences

This document describes the implemented application. It does not assert that every hackathon criterion has been independently benchmarked.

Product and user journey

Start with a request

Open the workspace and describe what the video should explain. The request may be a topic, an ordered business flow or an explanation grounded in an attached document. One chat represents one video-generation conversation.

```
Explain how a Python for loop works for a beginner in English in 30 seconds.
```

Confirm preferences

An editable card collects language, audience, target duration, visual style, captions, background sound and export resolution. Recognized preferences already present in the prompt are prefilled. Clarification choices use selectable cards instead of requiring the user to retype fixed answers.

Generate and watch

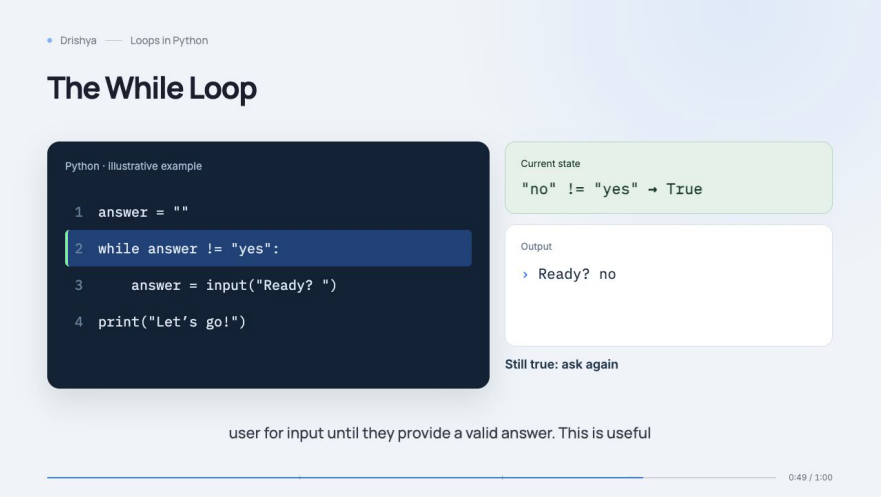
After confirmation, a durable background job plans the explanation, synthesizes speech and renders the video. Stage progress is saved. A narrated Remotion preview can be shown once the plan, narration and timeline are ready, while MP4 rendering continues.

Library and follow-ups

Ready videos are saved to the library automatically. A single video opens its player; a chapter collection opens its chapter selection. Follow-up requests create saved revisions and can reuse unchanged audio and visual segments. Previous versions remain accessible.

Duration and chapter behavior

Use 30-90 seconds for the hackathon lesson format. The broader app accepts longer requests. Chapters require a combined estimated duration greater than 120 seconds and user approval. Selected fixed durations are fitted to the final frame count; they are not fixed independently for every scene.



Actual local export: a Python while-loop demonstration with code, state, output and captions.

System architecture

The application separates the user interface, job coordination, local intelligence and rendering so a long generation does not run inside a frontend request.

Boundary	Responsibility
Next.js web app	Chat, preference cards, library, player, feedback and shared quizzes. Axios and React Query manage API state; Zustand manages UI state.
Node / Express API	Better Auth, ownership checks, uploads, revision records, job creation and media endpoints.
PostgreSQL + Drizzle	Accounts, conversations, sources, revisions, queue state, learning content, shares and results.
Generation worker	Claims saved jobs, reports progress, renews leases, runs local model calls and produces exports.
Ollama / Qwen3-8B	Structured lesson planning, narration text, visual direction, translation and learning content.
Python TTS service	Loads Indic Parler once and serializes narration requests using the existing environment.
Remotion + FFmpeg	Deterministic authored scenes, browser preview, frame rendering, audio assembly and MP4 verification.

Generation flow

```
Prompt / uploaded source
-> validated planning JSON
-> authored visual plan + narration text
-> local TTS -> measured WAV durations
-> audio-informed scene timeline
-> browser preview + local MP4 render
-> saved video, poster and captions
```

Data, not executable model code

Zod schemas validate model responses. The application supplies scene implementations and supported animation logic. It does not execute generated Python, JavaScript, HTML or SVG. Code snippets shown in lessons are text; Python-loop execution traces are authored demonstrations.

Visuals, narration and export

An engine of reusable scenes

Remotion renders React components from the validated visual plan. Scene families include labelled diagrams, code walkthroughs, arrays, comparisons, numeric charts, orbit views and spatial relationships. The UI theme is separate from the visual theme selected for a video.

Python-loop demonstrations

Supported Python-loop beats use a code-and-output view rather than a generic graph of labels. The trace highlights the active line, updates the current variable or condition, accumulates printed output and shows termination. Current authored examples cover list iteration, range-based repetition, a counter while loop and a repeated input condition. This is not a general Python interpreter.

Persistent local speech

The Node adapter calls an authenticated localhost TTS boundary. It invokes the existing Python 3.11 environment at ~/AI/indic-tts/.venv/bin/python. Model loading and generation reuse the known-working Indic Parler configuration. One loaded model serves successive requests; a lock serializes synthesis.

Narration language	Code / configured speaker
English	en / Thoma
Hindi	hi / Rohit
Kannada	kn / Suresh
Telugu	te / Prakash

These are the configured male speaker profiles. No hosted speech service or custom voice clone is required. Female selection and experimental Whisper verification are disabled.

Timing and media

Measured WAV durations inform scene lengths. Fixed-duration requests also use bounded script fitting, playback-rate adjustments and pauses. Captions are generally estimated narration chunks rather than word-level forced alignment. Review wording, pronunciation and timing before submission.

The standard export is 1280 x 720 at 30 fps with H.264 video and AAC audio. A 1080p preference scales the render. Exports include a poster and VTT/SRT captions. The renderer validates audio and the output media profile before marking an export complete. Background music and effects are generated procedurally in local code.

Installation and configuration

Prepare prerequisites while online

• Node.js 24+ and pnpm 11.1.3; workspace dependencies installed.

• Ollama with the exact configured tag downloaded, normally qwen3:8b.

• A working Python 3.11 Indic Parler environment with cached weights and both tokenizers.

• PostgreSQL tools: initdb, pg_ctl, createdb and psql.

• Installed Google Chrome, FFmpeg and FFprobe.

```
pnpm install --frozen-lockfile
ollama pull qwen3:8b
ollama list
```

Environment files

```
apps/api/.env
apps/web/.env.local
services/inference/.env
```

Copy the corresponding .env.example files only on a fresh setup. Preserve existing configured files. Live credentials are gitignored and must not be copied into frontend-public variables.

Setting	Purpose
DATABASE_URL	Standard PostgreSQL/Neon connection; offline preparation creates a separate local connection.
BETTER_AUTH_SECRET	Random value of at least 32 characters.
BETTER_AUTH_URL	http://localhost:3000 for local use.
OLLAMA_BASE_URL / MODEL	http://127.0.0.1:11434 / qwen3:8b
INFERENCE_URL / SECRET	http://127.0.0.1:8001; match the secret between API and speech service.
MEDIA_ROOT / DRISHYA_DATA_DIR	../../data in the standard API / speech configuration.
TTS_PYTHON	~/AI/indic-tts/.venv/bin/python
Web API_URL	http://127.0.0.1:4000

Do not recreate an already-working speech environment. The current launcher does not install packages or download weights. Legacy inference requirements files belong to the earlier implementation; the current README records the active environment inventory.

Offline operation

The offline profile runs the database, authentication, AI inference, speech and rendering on the same computer. Models, fonts, browser binaries and packages must already be present. No external AI API is used by the generation pipeline.

Prepare and start

```
pnpm offline:check
pnpm offline:prepare
# Stop the standard Drishya processes first.
pnpm dev:offline
```

Open http://localhost:3000 and register a local Drishya account. Offline accounts and media are separate from the standard Neon-backed profile. Existing online data is preserved and is not automatically migrated.

Service or storage	Local location
Web / API	localhost:3000 / 127.0.0.1:4000
Speech / Ollama	127.0.0.1:8001 / 127.0.0.1:11434
PostgreSQL	127.0.0.1:55432
Database cluster	data/postgres
Local credentials	.env.offline (mode 0600)
Offline media	data/offline

What startup does

The setup script reuses the dedicated PostgreSQL cluster and applies pending migrations. The offline startup command supplies consistent environment settings to the frontend, API, worker and TTS service. Endpoint checks require local addresses. Hugging Face model loading uses local_files_only with offline flags.

Disconnect and demonstrate

After preparation and startup, turn Wi-Fi off and keep using localhost. Downloaded MP4s play without the app or any model running. Missing model files and dependencies must be resolved during online preparation; generation does not download them on demand.

Stop or switch profiles

Stop development with Ctrl+C, then run **pnpm offline:stop**. To return to the standard profile, run **pnpm db:migrate** and **pnpm dev**. Do not run both profiles on the same ports. Neon requires internet; public share recipients must be able to reach the host application.

Documents, quizzes and sharing

Document-grounded explanations

A new chat accepts up to three text-based PDF, DOCX, TXT, Markdown or CSV files, each up to 10 MB. Extracted source text is limited to 12,000 characters across the conversation. Scanned documents require OCR outside the app. Original document binaries are not retained.

Uploads are saved as private extracted sources and bound to the owning conversation. The application passes documents separately from the user request, divides source text into exact passages and resolves model-selected passage IDs into original quotations. Source content is reference data, not executable instructions. Provenance checks are not a general fact-checking system.

Q&A; and quizzes

From a completed video in the library, request Q&A; and multiple-choice quiz content. A separate durable learning job generates content from the saved narration. Its failure or retry is independent of video rendering.

A quiz participant enters a name, answers the questions and submits an attempt. The server grades the submission and saves the result. Questions are frozen per attempt; correct answers are withheld until submission. Owners can view participant results. Participant names are self-reported.

Sharing controls

A share link can show Q&A;, show the quiz, show both or hide both. Public access does not require a Drishya account. Tokens are stored as hashes and links can be revoked. Media access is restricted to the shared video or its allowed chapters.

Language copies and feedback

Duplication creates a separate chat/video in the chosen language, preserving the source visual structure where possible and generating new narration as needed. Completed videos collect a one-to-five-star rating and an optional message. Feedback is persisted for future reporting; an admin dashboard is not included.

Privacy boundaries

Authenticated routes validate ownership of chats, documents, jobs, revisions and results. The private TTS service requires a shared bearer secret. Environment secrets and local media are excluded from Git. Share links expose only the content enabled for that share.

Persistence and operations

Durable work

Generation requests persist the target revision and queue record in PostgreSQL. The worker claims one heavy job at a time, renews its lease and records stage progress. Bounded retries, cancellation and recovery protect saved work without relying on an open browser tab.

Cache keys include the relevant text, speaker and generation settings. Changes can reuse unaffected speech and completed scene segments. Incomplete model streams and unfinished frame renders are restarted rather than resumed at an arbitrary token or frame.

Authentication

Better Auth uses the Drizzle PostgreSQL adapter for email/password accounts and database-backed sessions. Next.js proxies same-origin API requests to Express. No social providers, email verification or password-reset mail service are configured.

Repository map

```
apps/web Chat, library, player and shared quizzes
apps/api Auth, uploads, jobs and media endpoints
apps/worker Generation-worker launcher
services/inference Persistent Python TTS service
packages/ai Planner, translation and scene preparation
packages/contracts Shared Zod and timeline contracts
packages/db Drizzle schema and migrations
packages/motion Remotion scenes and execution traces
packages/ui Shared shadcn components and theme
scripts/offline.mjs Local database and offline startup
```

Useful checks

```
pnpm typecheck # Workspace TypeScript
pnpm check # Biome lint and formatting
pnpm db:check # Read-only standard DB connection check
pnpm db:generate # Generate schema migrations
pnpm db:migrate # Apply standard-profile migrations
```

Operational notes

Use pnpm dev:offline for the complete offline stack. Starting only the standard worker command does not automatically select the offline database. If an export is repaired in place, versioned media URLs and no-store headers help prevent an old browser download from being reused.

Generation failures show retry controls. The built-in example is available at /workspace/example; an automatic fully narrated fallback in the chat pipeline remains separate work. Chapter merging, OCR and a general arbitrary-code interpreter are outside the current implementation.

Demo, hardware and licences

Five-minute demonstration

• Start the prepared local profile and disable Wi-Fi.

• Type a new topic and confirm the preference card; use 30-90 seconds for the hackathon format.

• Show progress, play the narrated result and download the MP4.

• Open the library, show a reviewed language copy and demonstrate a follow-up or quiz.

The submission brief requests two topics, one judge-provided and one chosen by the team, each in English and one Indian language. Package the four reviewed exports separately; data/ is gitignored. The repaired English loop video is at data/offline/Drishya-Python-Loops-Repaired.mp4 on the demo machine.

Test environment

MacBook Pro, Apple M5 (Mac17,2), 16 GB unified memory; macOS 26.5.2; Node 24.15.0; pnpm 11.1.3; Python 3.11. Speech uses MPS on this machine. A CPU fallback exists, but a full CPU-only 8-16 GB benchmark and a modest-phone playback benchmark have not been recorded.

Historical fresh 30-second worker runs took approximately 238-304 seconds, with previews at 158-232 seconds. Prompts and intermediate code revisions varied. These are historical measurements, not a final-checkout latency guarantee. See docs/PERFORMANCE-REPORT.md for methodology.

Models and third-party assets

Component	Licence / attribution
Qwen3-8B	Apache-2.0; Qwen model weights.
Ollama	MIT; local model runtime.
AI4Bharat Indic Parler TTS	Apache-2.0 in the model card; built-in speaker profiles and repository access conditions apply.
Parler TTS implementation	Apache-2.0; Hugging Face implementation.
Remotion 4.0.539	Remotion Free/Company licence; eligibility and restrictions apply. Not MIT.
Bundled fonts	SIL OFL-1.1: Inter, Manrope, Instrument Serif, IBM Plex Mono and Noto Sans Indic families.
Lucide / FFmpeg	ISC / LGPL or GPL according to the FFmpeg build configuration.

Scenes and procedural sound are application-authored. No downloaded stock music or custom voice recordings are required. The repository currently has no project-level LICENSE file. Third-party licences do not assign a licence to Drishya source.

Source links

Qwen3-8B model card · Indic Parler model card
Remotion licence · FFmpeg licensing
Repository references: README.md, docs/V2-ARCHITECTURE.md, docs/PYTHON-LOOP-VISUALS.md and services/inference/README.md.